ПРЕЗЕНТАЦИЯ 3

Elixir

Конкурентность и отказоустойчивость

Процессы, BEAM, OTP и распределённые системы

Назарова Анастасия
Шилов Павел

Санкт-Петербургский политехнический университет Петра Великого
Высшая школа прикладной математики и вычислительной физики

2026 год

Конкурентность и параллелизм

Две разные концепции, которые дополняют друг друга

Конкурентность

Несколько задач могут находиться в процессе выполнения одновременно с точки зрения организации программы.

- Задачи чередуются между собой
- Не обязательно выполняются физически одновременно
- Позволяет эффективно использовать ресурсы

Параллелизм

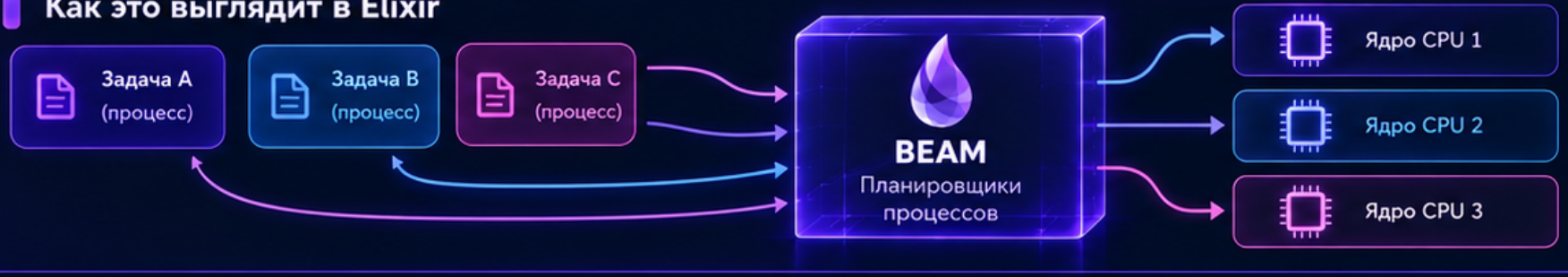
Несколько задач физически выполняются одновременно, например на разных ядрах процессора.

- Задачи выполняются одновременно
- Требует несколько ядер процессора
- Ускоряет общее время выполнения

В Elixir поддерживаются обе модели

- Лёгкие процессы BEAM позволяют запускать тысячи конкурентных задач
- Планировщики BEAM распределяют процессы по нескольким ядрам CPU
- Прозрачное масштабирование от конкурентности к параллелизму
- Подходит для высоконагруженных и отказоустойчивых систем

Как это выглядит в Elixir



Elixir обеспечивает конкурентность через лёгкие процессы BEAM и поддерживает параллелизм, выполняя их на нескольких ядрах процессора благодаря планировщикам виртуальной машины.

ПРЕЗЕНТАЦИЯ 3

Как работает виртуальная машина BEAM

Архитектура, процессы, планировщики и изоляция

Elixir код
Пишем код на Elixir
(компилируется в BEAM bytecode)

Компилятор
Код компилируется в
байткод BEAM
(.beam файлы)

Загрузка в BEAM
Модули загружаются
в виртуальную машину

Особенности BEAM

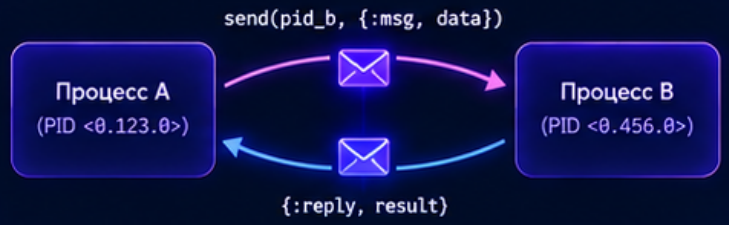
- **Лёгкие процессы**
  Тысячи процессов на одной виртуальной машине
- **Изоляция процессов**
  Каждый процесс имеет своё состояние и память
- **Планировщики**
  Распределяют процессы по ядрам CPU
- **Вытесняющее планирование**
  Обеспечивает справедливое выполнение
- **Автоматическое управление памятью**
  Отдельный GC для каждого процесса
- **Надёжность**
  Сбой одного процесса не влияет на другие
- **Масштабируемость**
  Эффективное использование многоядерных систем

Процесс в BEAM
Независимая единица выполнения

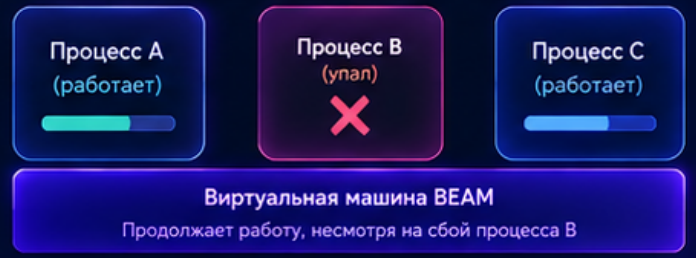
```
def loop do
  receive do
    {:ping, from} ->
      send(from, :pong)
      loop()
  end
end
```

- Собственный стек
- Собственная память
- Почтовый ящик
- Независимое выполнение
- Лёгкий (микроны памяти)
- Создаётся за микросекунды

Обмен сообщениями
Асинхронная модель без разделяемой памяти



Изоляция и надёжность
Сбой одного процесса не влияет на другие



ПРЕЗЕНТАЦИЯ 3

Создание процессов: spawn и PID

Процессы — это лёгкие изолированные единицы выполнения

Создание процесса

spawn/1 — запуск функции в новом процессе

```elixir
# Родительский процесс
parent = self()
IO.puts("Родительский  PID: #{inspect(parent)}")

# Создаём новый процесс
pid = spawn(fn ->
    IO.puts("Привет из нового процесса!")
    loop()
end)

IO.puts("Дочерний PID: #{inspect(pid)}")
```

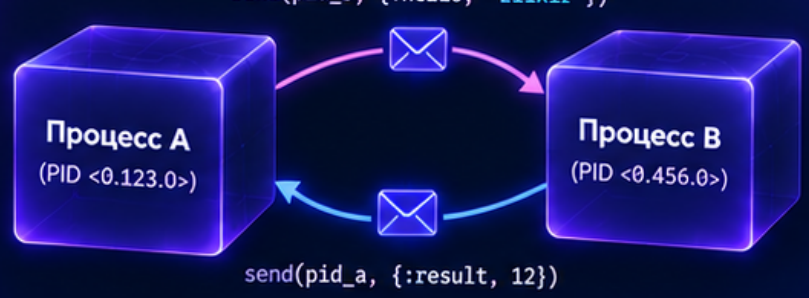
Результат выполнения

```
Родительский PID: #PID<0.123.0>
Дочерний PID: #PID<0.456.0>
Привет из нового процесса!
```

Что такое PID?

PID (Process Identifier) — уникальный идентификатор процесса в виртуальной машине BEAM.

Схема обмена сообщениями



Отправка сообщений

Процессы обмениваются сообщениями с помощью оператора !

```elixir
receiver = spawn(fn ->
    receive do
        {:hello, msg} ->
          IO.puts("Получено сообщение: #{msg}")
        {:add, a, b} ->
          send(self(), {:result, a + b})
    end
end)

# Отправляем сообщение
send(receiver, {:hello, "Elixir"})
send(receiver, {:add, 5, 7})
```

Свойства процессов в Elixir

- Изолированная память — нет общего состояния
- Очень лёгкие (тысячи процессов)
- Общаются только через сообщения
- Не разделяют переменные
- Надёжная изоляция сбоев
- Подходят для конкурентных и распределённых систем

ПРЕЗЕНТАЦИЯ 3

Mailbox и получение сообщений

Каждый процесс имеет собственную очередь сообщений

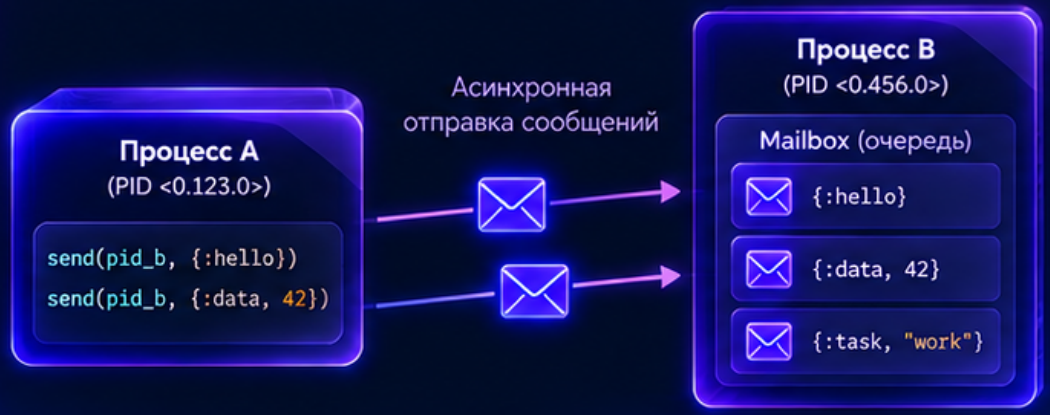
Основной механизм

Процессы общаются только через асинхронную отправку сообщений в mailbox.

```elixir
# Отправка сообщения
send(pid, {:msg, 123})

# Получение сообщения
receive do
    {:msg, value} ->
      IO.puts("Получено: #{value}")
after 5_000 ->
     IO.puts("Таймаут")
end
```

Как это работает

Сообщение помещается в mailbox процесса-получателя и обрабатывается, когда процесс его прочитает.



Особенности mailbox

- FIFO — сообщения обрабатываются в порядке отправки
- Неограниченная очередь (ограничивается только памятью)
- Каждое сообщение — произвольный терм в Elixir
- Отправка не блокирует отправителя
- Получение может быть селективным
- Поддержка таймаутов и шаблонов сопоставления

Практический пример

Простой сервер, который обрабатывает разные типы сообщений.

```elixir
def server do
    receive do
       {:ping, from} ->
          send(from, :pong)
       {:get_time, from} ->
          send(from, {:time, DateTime.utc_now()})
       {:stop, _from} ->
          IO.puts("Сервер завершает работу")
       _ ->
          IO.puts("Неизвестное сообщение")
    end
    server()  # продлажаем работать
end
```

Селективное получение

Можно обрабатывать только нужные сообщения, оставляя остальные в очереди.

```elixir
receive do
   {:important, data} ->
      IO.puts("Важное сообщение: #{inspect(data)}")
after 0 ->
    IO.puts("Важных сообщений нет")
end

# Остальные сообщения остаются в mailbox
receive do
   {:other, data} ->
      IO.puts("Обычное сообщение: #{inspect(data)}")
end
```

Таймауты и неблокирующее получение

Процесс не блокируется, если нет подходящих сообщений.

```elixir
# Неблокирующий приём
receive do
     {:msg, data} -> IO.puts(data)
after 0 ->
     IO.puts("Сообщений нет")
end
```

```elixir
# Приём с таймаутом
receive do
     {:msg, data} -> IO.puts(data)
after 5_000 ->
     IO.puts("Таймаут 5 секунд")
end
```

ПРЕЗЕНТАЦИЯ 3

Хранение состояния: setter и getter

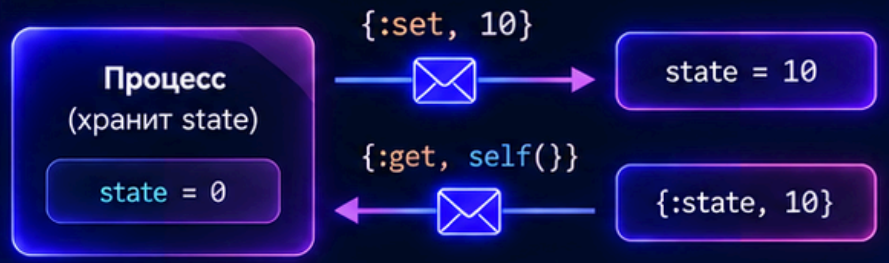
Как процесс может хранить данные и предоставлять к ним доступ

Идея хранения состояния

- Процесс хранит своё состояние во внутренней переменной.
- Состояние изменяется через сообщение `{:set, значение}`.
- Текущее состояние запрашивается сообщением `{:get, pid_ответа}`.
- Ответ отправляется обратно в виде `{:state, значение}`.

Как это работает

Процесс последовательно обрабатывает сообщения и обновляет своё состояние.



Особенности

- Состояние хранится только внутри одного процесса.
- Доступ к состоянию происходит через сообщения.
- Нет разделяемой памяти и блокировок.
- Можно реализовать любую логику обработки данных.
- Безопасный способ хранения данных в конкурентной среде.

Реализация процесса

Простой процесс, который поддерживает `{:set, ...}` и `{:get, ...}`.

```elixir
def loop(state) do
  receive do
    {:set, new_value} ->
      IO.puts("Устанавливаем состояние: #{new_value}")
      loop(new_value)

    {:get, from} ->
      send(from, {:state, state})
      loop(state)
  end
end
```

Пример использования

Два клиента: один устанавливает значение, другой запрашивает текущее состояние.

```elixir
pid = spawn(fn -> loop(0) end)

# Клиент 1 – устанавливает значение
send(pid, {:set, 10})

# Клиент 2 – запрашивает значение
send(pid, {:get, self()})

# Получаем ответ
receive do
  {:state, value} ->
    IO.puts("Текущее состояние: #{value}")
after 1_000 ->
    IO.puts("Ответ не получен")
end
```

Результат выполнения

Процесс хранит значение и корректно отвечает на запросы.

```
Устанавливаем состояние: 10
Текущее состояние: 10
```

Варианты развития

- Хранение более сложных структур (карты, списки).
- Обработка нескольких типов команд.
- Реализация счётчиков, кэшей, конфигураций.

ПРЕЗЕНТАЦИЯ 3

Link и Monitor: связь и отслеживание процессов

Как процессы узнают о завершении друг друга

Link — двусторонняя связь

Если один из связанных процессов завершается с ошибкой, то другой тоже падает (по умолчанию).

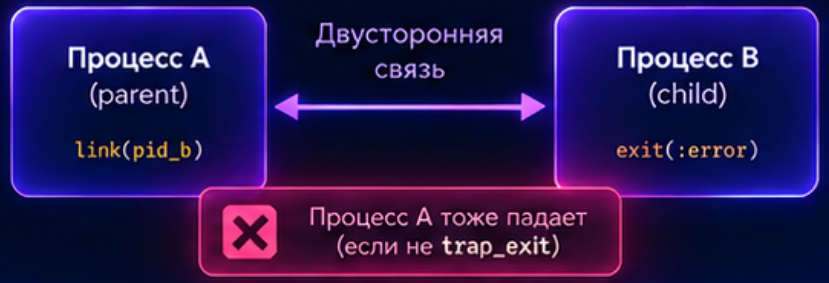
Пример: link

Два процесса связаны между собой.

```elixir
def parent do
  child = spawn(fn ->
    Process.link(self())
    :timer.sleep(1_000)
    exit(:error)
  end)

  Process.flag(:trap_exit, true)
  receive do
    {:EXIT, ^child, reason} ->
    IO.puts("Дочерний процесс упал: #{reason}")
  end
end
```

Поведение link при ошибке

Ошибка в одном процессе приводит к падению связанного процесса (если не перехватывать).



Особенности link

- Двусторонняя связь
- Ошибка в одном процессе может привести к падению другого
- Можно перехватывать с помощью `Process.flag(:trap_exit, true)`
- Используется для надёжной связки зависимых процессов

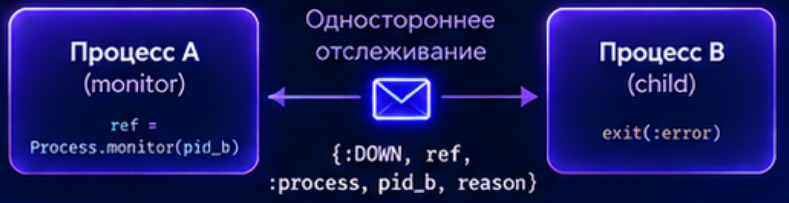
Monitor — одностороннее отслеживание

Процесс получает сообщение `{:DOWN, ...}`, но сам не падает.

```elixir
def monitor_example do
  pid = spawn(fn ->
    :timer.sleep(1_000)
    exit(:error)
  end)

  ref = Process.monitor(pid)
  receive do
    {:DOWN, ^ref, :process, ^pid, reason} ->
      IO.puts("Процесс завершился: #{reason}")
  end
end
```

Поведение monitor

Процесс получает уведдмление но продолжает работать.



Когда что использовать

- **Link** — для тесно связанных процессов, где падение одного должно останавливать другой
- **Monitor** — для наблюдения без падения (например, supervisor, watchdog, асинхронные задачи)

ПРЕЗЕНТАЦИЯ 3

Task и Agent: фоновые задачи и простой state

Удобные абстракции поверх процессов для простых сценариев

Task

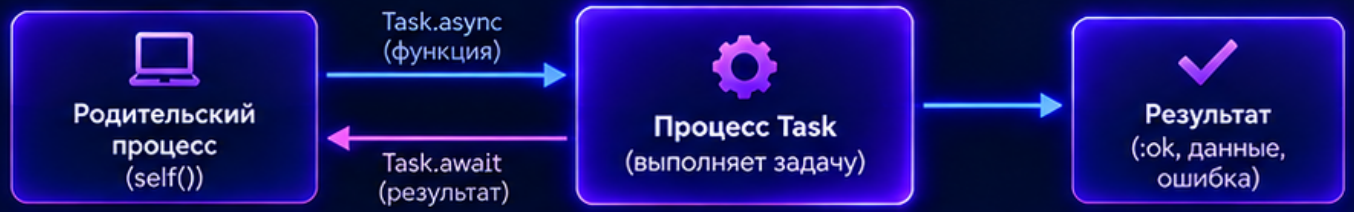
Выполнение одноразовых задач в отдельном процессе с простым получением результата.

Когда использовать Task

- Фоновая работа, которая не блокирует основной процесс
- Одноразовые вычисления и I/O
- Простое получение результата
- Подходит для параллельных задач

Как работает Task

Создаётся временный процесс, который выполняет задачу и возвращает результат.



Agent

Простое хранилище состояния во отдельном процессе. Автоматизирует get/set/update операции.

Когда использовать Agent

- Простое разделяемое состояние
- Не нужна сложная логика (в отличие от GenServer)
- Быстрые операции get/update
- Подходит для счётчиков, кэшей, настроек

Как работает Agent

Агент хранит состояние и позволяет безопасно его изменять через удобные функции.

Пример: Task

Запуск фоновой задачи и получение результата.

```elixir
task = Task.async(fn ->
  Process.sleep(1_000)
  :ok
end)

# где-то позже
result = Task.await(task)
IO.puts("Результат: #{inspect(result)}")
```

Пример: Agent

Создание агента, обновление и получение значения.

```elixir
# Создаём агента с начальным значением
agent = Agent.start_link(fn -> 0 end)

# Увеличиваем счётчик
Agent.update(agent, fn count -> count + 1 end)

# Получаем текущее значение
value = Agent.get(agent, & &1)
IO.puts("Текущее значение: #{value}")
```

ПРЕЗЕНТАЦИЯ 3

GenServer: надёжные процессы с состоянием

Стандартное поведение для долгоживущих процессов

Что такое GenServer

GenServer — это удобное поведение для создания процесса, который хранит состояние и безопасно обрабатывает сообщения.

Когда использовать GenServer

- Нужно хранить состояние
- Требуется безопасная обработка запросов
- Нужна надёжность и перезапуск при сбоях
- Подходит для сервисов, кэшей, счётчиков, менеджеров соединений

Особенности GenServer

- Хранит состояние в своём процессе
- Поддерживает синхронные и асинхронные запросы
- Автоматически перезапускается при сбое
- Интегрируется с supervision tree
- Широко используется в экосистеме Elixir
- Подходит для конкурентных и распределённых систем

Как это работает

Процесс получает сообщения, обрабатывает их в колбэках и изменяет состояние.

Жизненный цикл колбэков

init/1	— инициализация состояния
handle_call/3	— обработка синхронных запросов
handle_cast/2	— обработка асинхронных запросов
handle_info/2	— обработка прочих сообщений
terminate/2	— завершение процесса
code_change/3	— обновление кода на лету

Пример работы с консоли IEx

```
iex> {:ok, _} = Counter.start_link(0)
{:ok, #PID<0.123.0>}
iex> Counter.get()
0
iex> Counter.increment()
:ok
iex> Counter.get()
1
```

Пример: простой счётчик

GenServer, который хранит счётчик и поддерживает get/increment.

```elixir
defmodule Counter do
  use GenServer

  def start_link(initial \\ 0) do
    GenServer.start_link(__MODULE__, initial, name: __MODULE__)
  end

  # --- Клиентский API ---
  def get, do: GenServer.call(__MODULE__, :get)
  def increment, do: GenServer.cast(__MODULE__, :inc)

  # --- Колбэки ---
  def init(initial), do: {:ok, initial}

  def handle_call(:get, _from, state), do: {:reply, state, state}

  def handle_cast(:inc, state) do
    {:noreply, state + 1}
  end
end
```

ПРЕЗЕНТАЦИЯ 3

Supervisor: контроль и перезапуск процессов

Автоматическое восстановление при сбоях

Что такое Supervisor

Supervisor — это процесс из OTP, который следит за дочерними процессами и автоматически перезапускает их при сбоях.

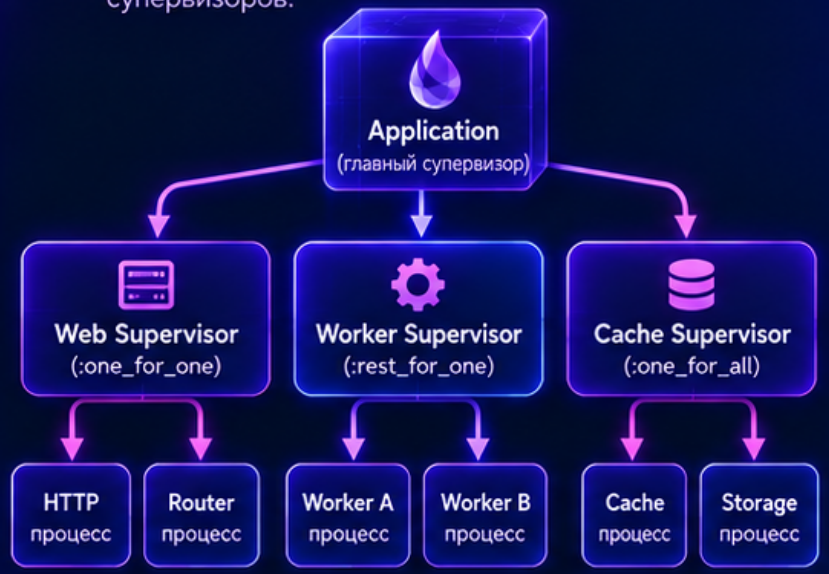
Пример супервизора

Простой супервизор, который запускает и перезапускает worker.

```elixir
defmodule MyApp.Supervisor do
  use Supervisor

  def start_link(init_arg) do
    Supervisor.start_link(__MODULE__, init_arg,
      name: __MODULE__)
  end

  def init(_arg) do
    children = [
      {MyApp.Worker, []}
    ]

    Supervisor.init(children,
      strategy: :one_for_one)
  end
end
```

Дерево надзора (supervision tree)

Иерархическая структура процессов под контролем супервизоров.



Пример дерева надзора

Реальная структура приложения.

Стратегии перезапуска

Встроенные стратегии OTP для разных сценариев.

:one_for_one	Перезапускается только упавший процесс
:rest_for_one	Перезапускается упавший процесс и все процессы, запущенные после него
:one_for_all	Перезапускаются все дочерние процессы при сбое одного
:simple_one_for_one	Подходит для динамически создаваемых процессов

Особенности Supervisor

- Автоматический перезапуск при сбоях
- Изоляция сбоев между процессами
- Иерархическая структура (дерево надзора)
- Различные стратегии перезапуска
- Интеграция с Application
- Подходит для долгоживущих сервисов

Что происходит при сбое

Supervisor перезапускает процессы автоматически.

```
[info] Worker started (PID <0.123.0>)
[error] Worker crashed: ** (RuntimeError) something went wrong
[info] Supervisor restarting MyApp.Worker...
[info] Worker started (PID <0.456.0>)
[info] System continues to work
```

ПРЕЗЕНТАЦИЯ 3

OTP: принципы и структура приложения

Подход к созданию надёжных, отказоустойчивых систем в Elixir

Что такое OTP

OTP — набор библиотек и принципов для создания надёжных, отказоустойчиивых и легко поддерживаемых приложений в Elixir.

Ключевые идеи

- Изоляция и перезапуск процессов
- Структурированная иерархия
- Наблюдение за сбоями
- Принцип «пусть падает» (let it crash)
- Стандартизированные поведенческие модули

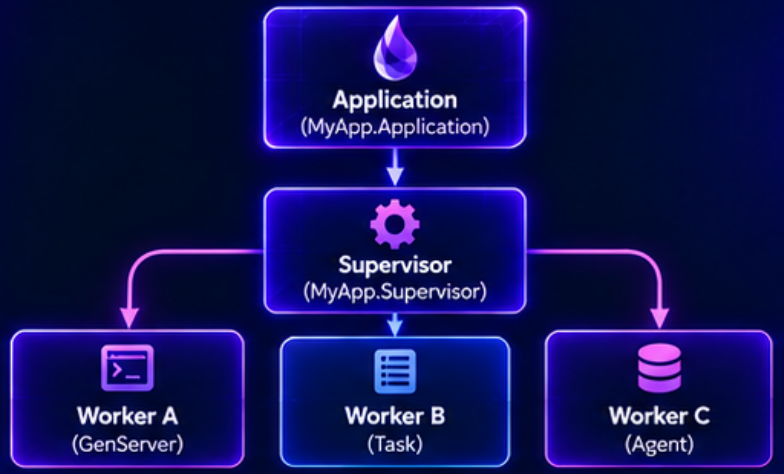
Структура OTP-приложения

Стандартная структура проекта.

```
my_app/
├─ lib/
│  └─ my_app/
│     ├─ application.ex   # точка входа
│     ├─ supervisor.ex    # корневой супервизор
│     ├─ worker.ex        # пример worker
│     └─ ...
├─ test/
├─ mix.exs
└─ config/
```

Иерархия процессов

Приложение строится в виде дерева супервизоров и дочерних процессов.



Пример супервизора

Запуск нескольких процессов под контролем OTP.

```elixir
defmodule MyApp.Supervisor do
  use Supervisor

  def start_link(_) do
    children = [
      {MyApp.Worker, []},
      {MyApp.Cache, []}
    ]

    Supervisor.start_link(children,
      strategy: :one_for_one, name: __MODULE__
    end)
end
```

Стратегии перезапуска

Определяют, как супервизор реагирует на сбой дочернего процесса.

`:one_for_one`	Перезапускается только упавший процесс
`:one_for_all`	Перезапускаются все дочерние процессы
`:rest_for_one`	Перезапускается упавший процесс и все процессы, запущенные после него
`:simple_one_for_one`	Подходит для динамически создаваемых процессов

Пример application.ex

Точка входа приложения.

```elixir
defmodule MyApp.Application do
  use Application

  def start(_type, _args) do
    children = [
      {MyApp.Supervisor, []}
    ]
    opts = [strategy: :one_for_one, name: MyApp.Supervisor]
    Supervisor.start_link(children, opts)
  end
end
```

Когда использовать OTP

- Долговижущие сервисы
- Нужна высокая надёжность и восстановление при сбоях
- Много параллельных процессов
- Подходит для продакшн систем и распределённых приложений

ПРЕЗЕНТАЦИЯ 3

Отказоустойчивость: изоляция и стратегия «let it crash»

Сбои — нормальная часть работы, и система проектируется так, чтобы их выдерживать

Философия «let it crash»

Процессы должны быть простыми и изолированными. При ошибке процесс завершается, а Supervisor автоматически перезапускает его.

Изоляция процессов

- Сбой одного процесса не влияет на другие
- Каждый процесс имеет своё состояние
- Нет общего состояния — нет гонок данных
- Ошибки не распространяются по системе

Пример сбоя процесса

Процесс падает из-за ошибки, но система продолжает работать.

```elixir
defmodule Worker do
  def start_link(id) do
    Task.start_link(fn ->
      IO.puts("Worker #{id} запущен")
      raise "Пример ошибки!"
    end)
  end
end

# Запускаем процесс, который упадёт
{:ok, pid} = Worker.start_link(1)
Process.sleep(100)
IO.puts("Система продолжает работать")
```

Как изоляция защищает систему

Падение одного процесса не останавливает другие процессы.

Сообщения при завершении

При использовании monitor процесс-наблюдатель получает сообщение :DOWN.

```
{:DOWN, ref, :process, pid, reason}
```

Ссылка монитора · Тип · PID упавшего процесса · Причина завершения

Сообщения о сбое

При падении процесса другие процессы могут получить уведомление.

```elixir
defmodule MonitorExample do
 def start do
   parent = self()
   spawn(fn ->
     send(parent, {:ready, self()})
     raise "Intentional crash"
   end)

   receive do
     {:ready, pid} ->
       Process.monitor(pid)
       IO.puts("Наблюдаем за процессом: #{pid}")
     {:DOWN, ref, :process, pid, reason} ->
       IO.puts("Процесс #{pid} завершился: #{inspect(reason)}")
   end
end
```

Лучшие практики

- Не перехватывать все ошибки (не использовать rescue повсеместно)
- Использовать Supervisor для автоматического перезапуска
- Делать процессы как можно более простыми
- Изолировать потенциально нестабильную логику
- Логировать ошибки и анализировать причины
- Использовать мониторинг для важнейших процессов

ПРЕЗЕНТАЦИЯ 3

Управление памятью и Garbage Collector

Каждый процесс BEAM имеет свою память и собственный сборщик мусора

Память процессов

Каждый процесс в BEAM имеет собственное изолированное виртуальное адресное пространство (heap). Процессы не разделяют память, что делает систему устойчивой к сбоям и упрощает сборку мусора.

Структурное разделение данных

В Elixir используются неизменяемые структуры данных (immutable). При изменении создаётся новая версия, которая переиспользует существующие части (structural sharing), а не копирует всё целиком.

Пример: добавление в начало списка за O(1)

Добавление элемента в начало списка — очень быстрая операция. Создаётся только новый «узел», который ссылается на старый список.

```elixir
# Исходный список
list = [2, 3, 4]

# Новый список с добавленным элементом
new_list = [1 | list]

IO.inspect(list)      # [2, 3, 4]
IO.inspect(new_list)  # [1, 2, 3, 4]
```

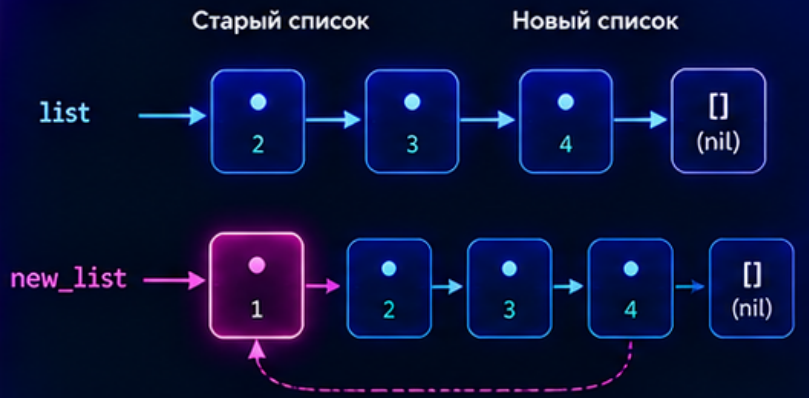
Как работает сборщик мусора (GC)

У укждого процесса свой лёгкий и быстрый GC, который останавливает только этот процесс. Другие процессы продолжают работать.

1. Процесс выделяет память на своём heap
2. При достижении порога запускается GC
3. Копируются живые объекты (generational)
4. Неживые объекты освобождаются
5. Процесс продолжает работу

Минимальные паузы и предсказуемое время работы

Как работает structural sharing



Новый узел. Остальные элементы переиспользуются

Особенности GC в BEAM

- Сборка мусора на уровне отдельных процессов
- Короткие паузы (не влияют на другие процессы)
- Генерационное поколение (young/old)
- Подходит для высококонкурентных систем
- Нет глобальной остановки системы (stop-the-world)
- Автоматическое управление памятью

Преимущества для высоконагруженных систем

- Эффективное использование памяти
- Меньше копирования данных
- Высокая производительность операций
- Хорошо масштабируется при большом количестве процессов
- Стабильная работа под нагрузкой

Пример с картой (map)

При обновлении создаётся новая версия карты, но неизменённые части переиспользуются.

```elixir
user = %{name: "Pavel", age: 20}
updated_user = %{user | age: 21}

IO.inspect(user)         # %{name: "Pavel", age: 20}
IO.inspect(updated_user) # %{name: "Pavel", age: 21}
```

ПРЕЗЕНТАЦИЯ 3

Распределённые системы в Elixir/Erlang

Связь узлов, обмен сообщениями и построение отказоустойчивых кластеров

Зачем распределённые системы

Позволяют объединять несколько BEAM-узлов в кластер, масштабировать приложение, обеспечивать отказоустойчивость и распределять нагрузку.

- Горизонтальное масштабирование
- Высокая доступность
- Распределённая обработка задач
- Автоматическое восстановление после сбоев

Как узлы находят друг друга

Узлы соединяются по протоколу Erlang distribution и обмениваются сообщениями. Для авторизации используется общий секрет — Erlang cookie.

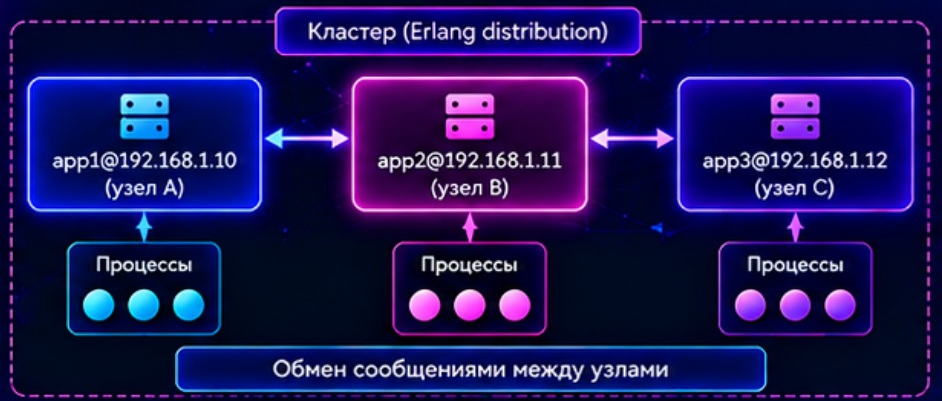
- Одинаковый cookie на всех узлах
- Уникальные имена узлов
- Доступ к EPMD (порт 4369) и настроенному TCP-порту Erlang Distribution
- Сетевой доступ между узлами

Пример: подключение к другому узлу

```elixir
# Текущее имя узла
Node.self()       # => :"app1@192.168.1.10"

# Подключение к другому узлу
Node.connect(:"app2@192.168.1.11")
# => true | false

# Список подключённых узлов
Node.list()
      # => [:"app2@192.168.1.11"]
```

Пример распределённого кластера

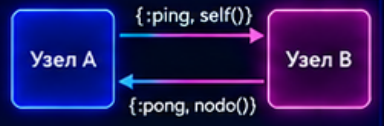
Несколько узлов, объединённых в кластер, могут обмениваться сообщениями и выполнять задачи на разных машинах.



Обмен сообщениями между узлами

Процессы на разных узлах взаимодействуют так же, как локальные процессы — с помощью сообщений (send/receive).

```elixir
# Запуск процесса на другом узле и обмен сообщениями
remote_pid = Node.spawn(:"app2@192.168.1.11", fn ->
  receive do
    {:ping, from} ->
      send(from, {:pong, node()})
  end
end)

# Отправка сообщения удалённому процессу
send(remote_pid, {:ping, self()})

# Получение ответа
receive do
  {:pong, node_name} ->
    IO.puts("Ответ от узла: #{node_name}")
end
```



- Тот же send/receive, что и локально
- Процессы могут быть на разных узлах
- Сообщения асинхронные
- Используется сетевое соединение Erlang Distribution

Регистрация процессов

Для удобного взаимодействия между узлами можно использовать глобальную регистрацию или через `:via`.

```elixir
# Локальная регистрация (на текущем узле)
Process.register(self(), :worker)

# Глобальная регистрация (во всём кластере)
:global.register_name(:worker, self())

# Получение PID удалённого процесса
:global.whereis_name(:worker)
#=> #PID<...> или :undefined
```

Обработка разъединения узлов

Можно отслеживать подключение и отключение узлов и реализовывать логику восстановления задач.

```elixir
defmodule NodeMonitor do
  use GenServer

  def start_link(_arg) do
    GenServer.start_link(__MODULE__, :ok, name: __MODULE__)
  end

  def init(_arg) do
    :net_kernel.monitor_nodes(true)
    {:ok, %{}}
  end
  def handle_info({:nodedown, node}, state) do
    IO.puts("Узел #{node} отключился")
    # Здесь можно перезапустить задачи, пересоздать соединения и т.д.
    {:noreply, state}
  end
```

Лучшие практики

- Использовать короткие имена узлов в локальной сети
- Одинаковый Erlang cookie на всех узлах
- Настроить сетевой доступ и EPMD (порт 4369)
- Использовать Supervisor для автоматического восстановления
- Не хранить критичные данные только на одном узле
- Тестировать поведение при потере узла
- Логировать события кластера

ПРЕЗЕНТАЦИЯ 3

Elixir + BEAM + OTP: итоговая схема работы

Шесть ключевых механизмов, которые вместе обеспечивают надёжность и масштабируемость

1. Processes

Лёгкие изолированные процессы с собственным heap и почтовым ящиком. Изоляция процессов ограничивает последствия сбоев и повышает надёжность.

2. Messages

Асинхронная передача сообщений между процессами (внутри узла и между узлами). Нет общих данных и блокировок, что упрощает разработку конкурентных систем.

3. Garbage Collector

Локальная сборка мусора в каждом процессе. Обычно не останавливает остальные процессы BEAM.
Короткие паузы и предсказуемое время работы, что помогает выдерживать нагрузку.

Elixir + BEAM + OTP

Надёжная платформа для создания отказоустойчивых, масштабируемых приложений

4. GenServer

Стандартная абстракция для написания надёжных процессов. Единый интерфейс и предсказуемое поведение.
Упрощает разработку, поддержку и расширение приложения.

5. Supervisor

Наблюдает за процессами, автоматически перезапускает их при сбоях и применяет стратегии (one_for_one, one_for_all, rest_for_one). Обеспечивает устойчивую работу системы.

6. Nodes

Распределённые системы из нескольких узлов. Обмен сообщениями между узлами, глобальная регистрация имён. Обнаружение потери соединений (nodedown/nodedup), восстановление реализуется архитектурой приложения.

Отказоустойчивость — результат архитектуры выполнения приложения

Процессы, обмен сообщениями, сборка мусора, GenServer, supervision и распределённые узлы работают вместе, обеспечивая надёжность, масштабируемость и устойчивость к сбоям.

ПРЕЗЕНТАЦИЯ 3

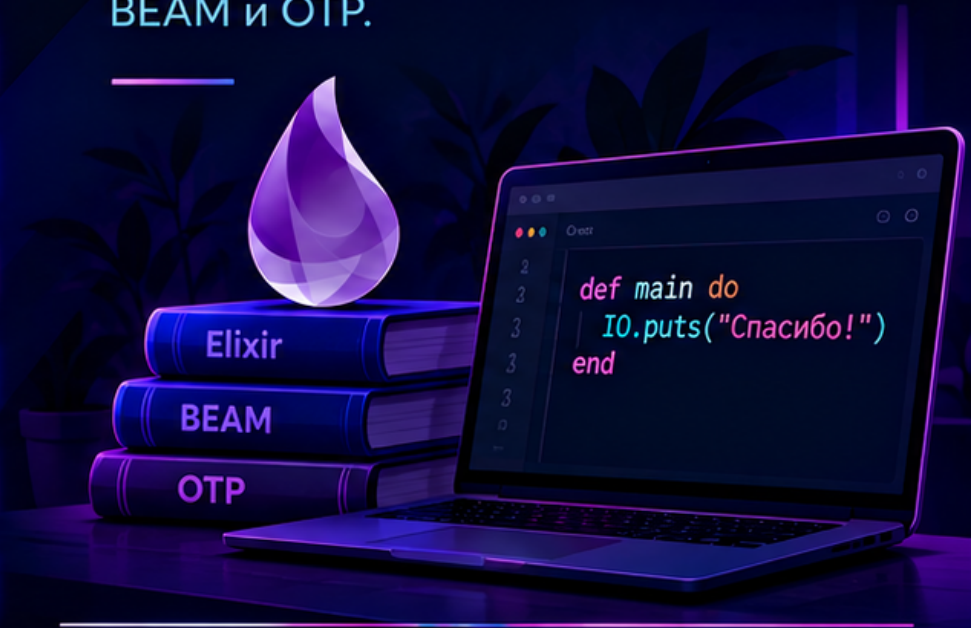
Литература и материалы

Официальные источники и полезные ресурсы по Elixir, BEAM и OTP

1. **Elixir — официальное руководство** Официальная документация, основы языка, стандартная библиотека, лучшие практики.	https://hexdocs.pm/elixir/main.html
2. **Processes** Документация по процессам в Elixir.	https://hexdocs.pm/elixir/Process.html
3. **Messages** Обмен сообщениями между процессами.	https://hexdocs.pm/elixir/main.html#messages
4. **Task** Параллельные задачи и удобные абстракции.	https://hexdocs.pm/elixir/Task.html
5. **Agent** Простое состояние для конкурентных приложений.	https://hexdocs.pm/elixir/Agent.html
6. **GenServer** Стандартная абстракция для надёжных процессов.	https://hexdocs.pm/elixir/GenServer.html
7. **Supervisor** Надзор за процессами и стратегии перезапуска.	https://hexdocs.pm/elixir/Supervisor.html
8. **Application** Структура приложений, запуск и конфигурация.	https://hexdocs.pm/elixir/Application.html
9. **Node** Распределённые системы и работа между узлами.	https://hexdocs.pm/elixir/Node.html
10. **Erlang/OTP** Официальная документация Erlang/OTP (для понимания BEAM, supervision, distribution).	https://www.erlang.org/docs

Спасибо за внимание!

Надеемся, что презентация помогла понять, как Elixir сочетает простоту разработки с надёжной платформой BEAM и OTP.



Назарова Анастасия | Шилов Павел | СПбПУ 2026 год